COMP 3920 - Database Systems 1

Relationship Models and Implementation

Required One-to-One

Not Enforceable directly in database. Required One-to-One implies data be entered into both tables at the same time, which is not possible; you must put data into each table one at a time.

Use Optional One-to-One instead.

Optional One-to-One

Canadian Passport.
Business Rules:
A person can only have 1 Canadian passport (but they don't have to have one).
A passport must belong to only 1 person.

Good Design:

Order of table creation and data entry
1st

Person

PK	person_id
	first_name
	last_name

Passport

PK	passport_id
FK, UQ, NN	person_id
	issue_date
	expiry_date

Order of table creation and data entry
2nd

PK = Primary Key, **FK** = Foreign Key
UQ = Unique, **NN** = Not Null

The good design enforces person to exist ***before*** passport can exist so that no passport can be created without an owner.

Bad Design:

Order of table creation and data entry
2nd

Person

PK	person_id
FK, UQ, NN	passport_id
	first_name
	last_name

Passport

PK	passport_id
	issue_date
	expiry_date

Order of table creation and data entry
1st

PK = Primary Key, **FK** = Foreign Key
UQ = Unique, **NN** = Not Null

This bad design allows (in fact requires) passport to be created ***before*** person allowing unowned passports!

Doubly Optional One-to-One

Person

Order of table creation and data entry
1st
(optional)

PK	person_id
	first_name
	last_name

Passport

PK	passport_id
FK, UQ	person_id
	issue_date
	expiry_date

Order of table creation and data entry
2nd
(optional)

PK = Primary Key, **FK** = Foreign Key
UQ = Unique, **NN** = Not Null

Person

Order of table creation and data entry
2nd
(optional)

PK	person_id
FK, UQ	passport_id
	first_name
	last_name

Passport

PK	passport_id
	issue_date
	expiry_date

Order of table creation and data entry
1st
(optional)

PK = Primary Key, **FK** = Foreign Key
UQ = Unique, **NN** = Not Null

This example is where the person_id foreign key (FK) is allowed to be null. This is shown for demonstrating the doubly-optional one-to-one; it would not be recommended (bad design) for the Person-Passport relationship. Again, this would allow a passport to be created ***before*** the person and thus have no owner.

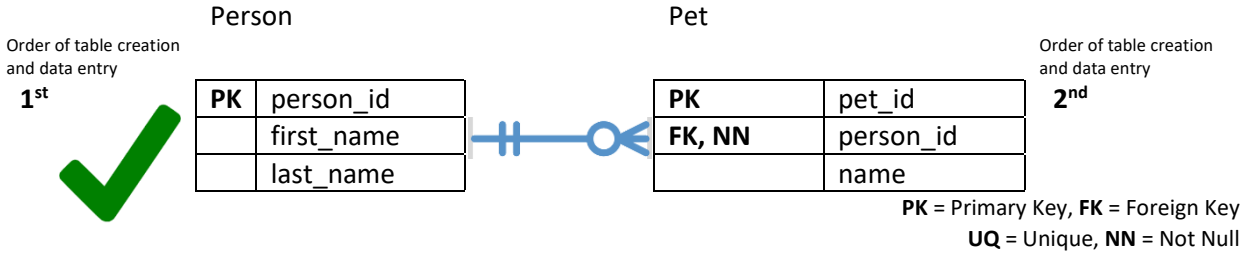
Required One-to-Many

Not Enforceable directly in database. Required One-to-Many implies data be entered into both tables at the same time, which is not possible; you must put data into each table one at a time.

Use Optional One-to-Many instead.

Optional One-to-Many

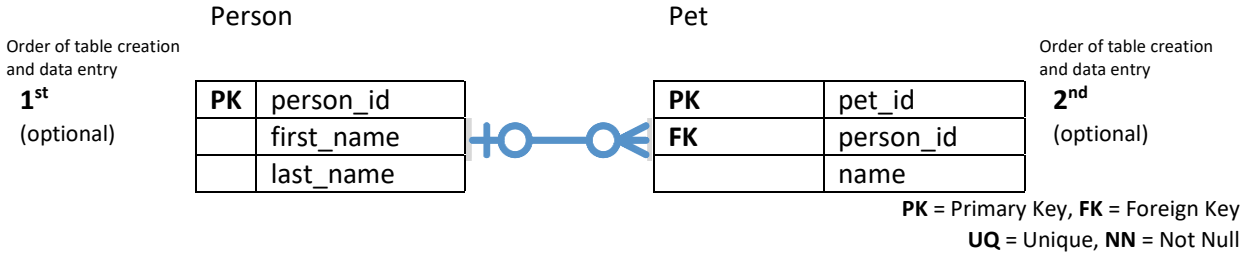
People and their Pets.
Business Rules:
A person can have many pets (but they don't have to have one).
A pet must belong to only 1 person.



Person must exist ***before*** the pet.

Doubly Optional One-to-Many

People and their Pets.
Business Rules:
A person can have many pets (but they don't have to have one).
A pet can belong to 1 person, but in this case don't have to and could be considered a stray (no known owner).



Pets can exist without people (strays).

Required Many-to-Many

Not Enforceable directly in database. Required Many-to-Many implies data be entered into both tables at the same time, which is not possible; you must put data into each table one at a time.

See Optional Many-to-Many.

Optional Many-to-Many

A Many-to-Many is not ***directly*** enforceable. It is, however, enforceable if split into multiple Optional One-to-Many relationships.

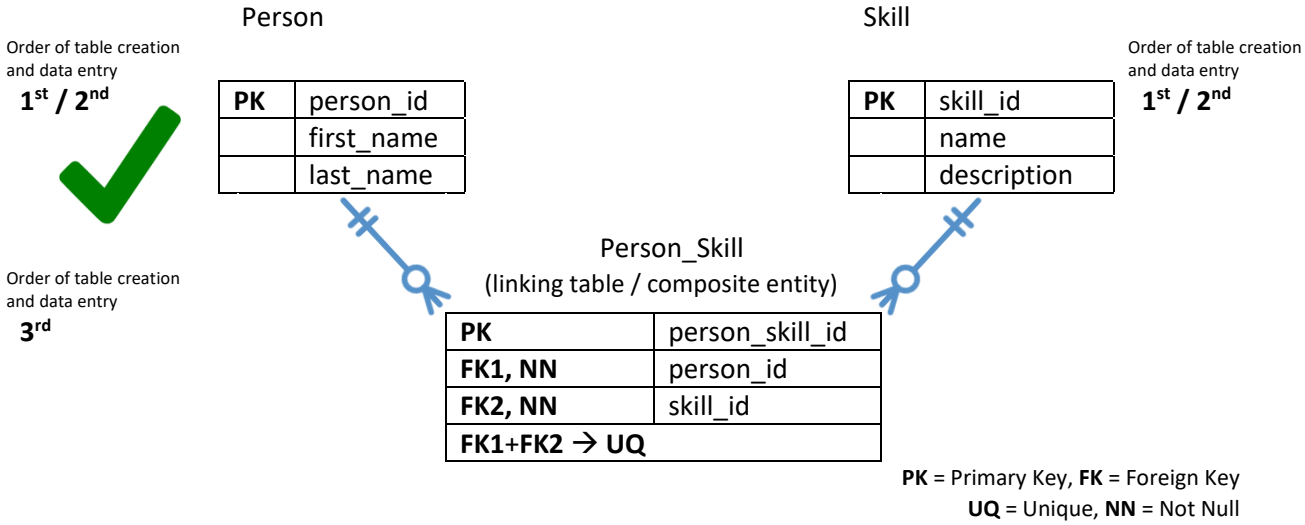
So this:

Becomes this:

and

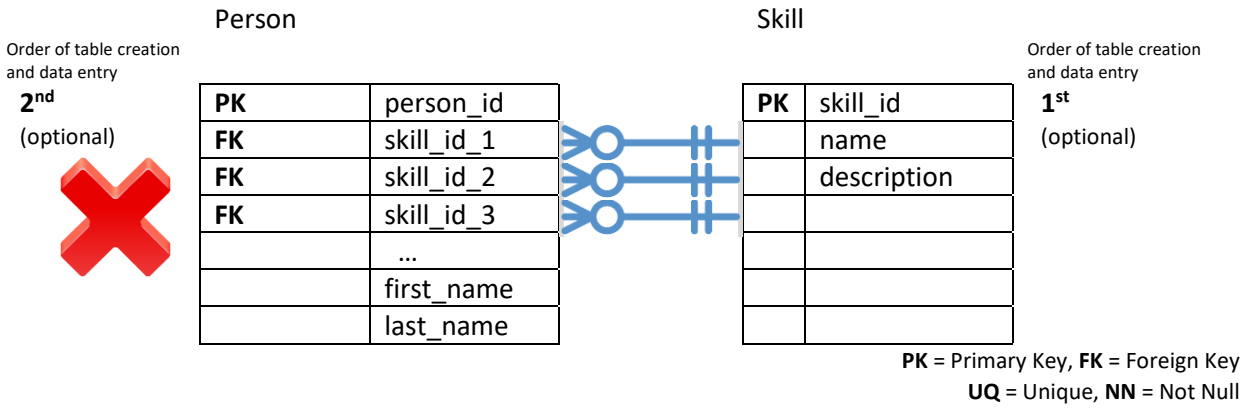
Person and Skills.
Business Rules:
A person can have may skills.
Skills can be mastered by many people.

Good Design:



The good design creates an intermediate linking table called an associative entity or composite entity.

Bad Design:



This bad design creates a new column in the table for every skill each person might have. This causes a number of issues. First, how many skills can people have? 10? 100? More? Do we really want to have a table with 100 (or more) columns? Second, for those who don't have as many skills as others, all those columns will be left NULL and be wasted space in the database.